



โครงงาน

จำแนกสายพันธุ์สุนัข





การจัดการข้อมูล

1. การคัดกรองและลบข้อมูลที่เป็น Noise เช่น
    - ภาพไม่คมชัด
    - ภาพที่มีบุคคลบังสุนัข
    - ภาพที่มีสุนัขมากกว่า 1 ตัว
    - ภาพที่สุนัขสวมเครื่องประดับ, เสื้อผ้า, อื่นๆ

โดยคัดเลือกและปรับจำนวนข้อมูลให้คงเหลือสายพันธุ์ละ 100 ภาพ เท่านั้น



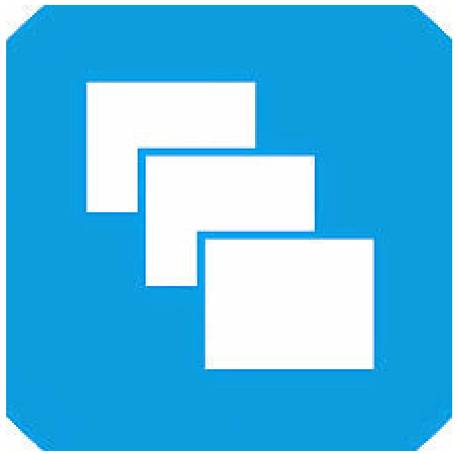






การจัดการข้อมูล

2. การตรวจหาภาพซ้ำ (Duplicate Detection)
    - ใช้โปรแกรม AllDup เพื่อตรวจสอบภาพซ้ำ
    - พบความซ้ำซ้อนในโฟลเดอร์:
        - coated_retriever
        - flat-coated_retriever



จึงดำเนินการลบโฟลเดอร์ coated_retriever ออกจากชุดข้อมูล ส่งผลให้ชุดข้อมูลสุดท้ายประกอบด้วยสายพันธุ์สุนัขจำนวน 119 สายพันธุ์ รวม 11,900 ภาพ

จำนวนข้อมูล

ข้อมูลที่ใช้ในโครงงานเป็นภาพถ่ายสุนัขจำนวน 119 สายพันธุ์ สายพันธุ์ละ 100 รูป
รวมทั้งหมด 11,900 รูป
ชุดข้อมูลถูกแบ่งออกเป็น

- ชุดข้อมูลฝึก (Training set) 70%
- ชุดข้อมูลตรวจสอบ (Validation set) 20%
- ชุดข้อมูลทดสอบ (Test set) 10%





การจำแนกชุดข้อมูลสำหรับ Training Set, Validation และ Test Set

```python
import os

splits = ['train', 'valid', 'test']

for split in splits:
    for breed in breeds:
        os.makedirs(os.path.join(output_path, split, breed), exist_ok=True)
```

1. สร้างโครงสร้างโฟลเดอร์สำหรับเตรียมเก็บข้อมูล Training Set, Validation และ Test Set

```python
import os
import random
import shutil

raw_path = '/content/drive/MyDrive/Project73-Dag/Images'
output_path = '/content/drive/MyDrive/Project73-Dag/dog_split'

splits = ['train', 'valid', 'test']
num_per_split = {'train':70, 'valid':20, 'test':10}
breeds = os.listdir(raw_path)

for split in splits:
    for breed in breeds:
        os.makedirs(os.path.join(output_path, split, breed), exist_ok=True)

for breed in breeds:
    breed_path = os.path.join(raw_path, breed)
    images = os.listdir(breed_path)
    random.shuffle(images)

    start = 0
    for split in splits:
        count = num_per_split[split]
        selected_images = images[start:start+count]
        start += count

        for img in selected_images:
            src = os.path.join(breed_path, img)
            dst = os.path.join(output_path, split, breed, img)
            shutil.copy(src, dst)
```

2. การจัดเตรียมและแบ่งชุดข้อมูลอัตโนมัติ

- ระบุโฟลเดอร์ต้นทาง และตั้งค่าจำนวนรูปภาพที่ต้องการแบ่ง (70/20/10)
- ให้ระบบสร้างโฟลเดอร์แยกตามกลุ่มใช้งานและแยกตามสายพันธุ์สุนัขให้โดยอัตโนมัติ
- นำรูปภาพทั้งหมดมาสลับลำดับแบบสุ่มเพื่อให้โมเดลได้เรียนรู้ข้อมูลที่หลากหลาย
- คัดลอกรูปภาพลงในโฟลเดอร์ที่เตรียมไว้ตามจำนวนที่ระบุอย่างแม่นยำ

การตรวจสอบจำนวนข้อมูลหลังการแบ่งชุดข้อมูล

```
1 for split in splits:
2     split_path = os.path.join(output_path, split)
3     total = sum(len(os.listdir(os.path.join(split_path, breed))) for breed in os.listdir(split_path))
4     print(f"{split} set: {total} images")
```

```
train set: 8330 images
valid set: 2380 images
test set: 1190 images
```

- วนลูปทีละชุดข้อมูล (train, valid, test) เพื่อเข้าถึงโฟลเดอร์ของแต่ละชุด
- สำหรับแต่ละชุด จะรวมจำนวนภาพจากทุกโฟลเดอร์สายพันธุ์ด้วยการนับไฟล์ภายในโฟลเดอร์
- แสดงผลจำนวนภาพทั้งหมดในแต่ละชุดข้อมูลอย่างเป็นสรุป

มี train set จำนวน 8330 ภาพ validation set จำนวน 2380 ภาพ มี test set จำนวน 1190 ภาพ



การสร้างโมเดล MobileNetV2

```python
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras import layers, models

base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
base_model.trainable = False

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.2),
    layers.Dense(119, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

print("สร้างโครงสร้างโมเดลเสร็จเรียบร้อย")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_
9406464/9406464 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
สร้างโครงสร้างโมเดลเสร็จเรียบร้อย
```

1. การสร้างโมเดล

- เลือกใช้โมเดล MobileNetV2 ที่ผ่านการฝึกจาก ImageNet เป็นโมเดลพื้นฐาน
- แช่แข็งพารามิเตอร์ของโมเดลพื้นฐานเพื่อใช้ในการดึงคุณลักษณะจากภาพ
- เพิ่มเลเยอร์ Global Average Pooling และ Dropout เพื่อช่วยลด Overfitting
- กำหนดเลเยอร์สุดท้ายสำหรับจำแนกภาพจำนวน 119 คลาส
- ตั้งค่าการคอมไพล์โมเดลด้วย Adam และ Categorical Crossentropy
- ใช้ Accuracy เป็นตัวชี้วัดประสิทธิภาพของโมเดล

การสร้างโมเดล MobileNetV2

```python
from tensorflow.keras.callbacks import ModelCheckpoint

checkpoint_path = '/content/drive/MyDrive/Project73-Dag/dog_breed_model_v2.h5'

checkpoint = ModelCheckpoint(
    checkpoint_path,
    monitor='val_accuracy',
    save_best_only=True,
    mode='max',
    verbose=1
)

history = model.fit(
    train_generator,
    validation_data=valid_generator,
    epochs=10,
    callbacks=[checkpoint]
)

print(f"เทรนเสร็จสมบูรณ์! ไฟล์โมเดลอยู่ที่: {checkpoint_path}")
```

2. การฝึกโมเดล

- กำหนดตำแหน่งสำหรับบันทึกไฟล์โมเดลลงใน Google Drive
- ใช้ Model Checkpoint เพื่อบันทึกเฉพาะโมเดลที่ให้ค่า Validation Accuracy สูงที่สุด
- ฝึกโมเดลเป็นจำนวน 10 Epoch พร้อมตรวจสอบผลด้วย Validation Set
- ระบบบันทึกไฟล์โมเดลที่ดีที่สุดโดยอัตโนมัติสำหรับนำไปใช้งานภายหลัง



ผลลัพธ์การสร้างโมเดล MobileNetV2

```
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should c
  self._warn_if_super_not_called()
Epoch 1/10
261/261 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.3981 - loss: 2.7188
Epoch 1: val_accuracy improved from -inf to 0.76577, saving model to /content/drive/MyDrive/Project73-Dag/dog_breed_model_v2.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered l
261/261 ━━━━━━━━━━━━━━━━━━━━ 519s 2s/step - accuracy: 0.3988 - loss: 2.7148 - val_accuracy: 0.7658 - val_loss: 0.8163
Epoch 2/10
261/261 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.8358 - loss: 0.5734
Epoch 2: val_accuracy improved from 0.76577 to 0.77502, saving model to /content/drive/MyDrive/Project73-Dag/dog_breed_model_v2.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered l
261/261 ━━━━━━━━━━━━━━━━━━━━ 469s 2s/step - accuracy: 0.8357 - loss: 0.5734 - val_accuracy: 0.7750 - val_loss: 0.7171
Epoch 3/10
261/261 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.8923 - loss: 0.3829
Epoch 3: val_accuracy improved from 0.77502 to 0.79016, saving model to /content/drive/MyDrive/Project73-Dag/dog_breed_model_v2.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered l
261/261 ━━━━━━━━━━━━━━━━━━━━ 465s 2s/step - accuracy: 0.8923 - loss: 0.3829 - val_accuracy: 0.7902 - val_loss: 0.6692
Epoch 4/10
261/261 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.9309 - loss: 0.2775
Epoch 4: val_accuracy did not improve from 0.79016
261/261 ━━━━━━━━━━━━━━━━━━━━ 497s 2s/step - accuracy: 0.9309 - loss: 0.2775 - val_accuracy: 0.7897 - val_loss: 0.6587
Epoch 5/10
261/261 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.9500 - loss: 0.2108
Epoch 5: val_accuracy improved from 0.79016 to 0.79815, saving model to /content/drive/MyDrive/Project73-Dag/dog_breed_model_v2.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered l
261/261 ━━━━━━━━━━━━━━━━━━━━ 447s 2s/step - accuracy: 0.9500 - loss: 0.2109 - val_accuracy: 0.7981 - val_loss: 0.6486
Epoch 6/10
261/261 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.9662 - loss: 0.1603
Epoch 6: val_accuracy did not improve from 0.79815
```

ความแม่นยำบนชุดตรวจสอบ (Validation Accuracy) สูงสุดที่ประมาณ 79.8%

- โมเดลให้ผลดีที่สุดในช่วง Epoch ที่ 5
- หลังจากนั้น Validation Accuracy ไม่เพิ่มขึ้น แสดงแนวโน้มเริ่มเกิด Overfitting
- ระบบได้บันทึกโมเดลที่ให้ผลดีที่สุดไว้สำหรับการนำไปทดสอบและใช้งานต่อ

การสร้างโมเดล InceptionV3

```python
from tensorflow.keras.applications import InceptionV3
from tensorflow.keras import layers, models

base_model = InceptionV3(weights='imagenet', include_top=False, input_shape=(299, 299, 3))

base_model.trainable = False

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(119, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

print("สร้างโมเดลสำเร็จแล้ว")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/inception_v3/inception_v3_
87910968/87910968 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
สร้างโมเดลสำเร็จแล้ว
```

1. การสร้างโมเดล

- เลือกใช้โมเดล InceptionV3 ที่ผ่านการฝึกจาก ImageNet เป็นโมเดลพื้นฐาน
- แช่แข็งพารามิเตอร์ของโมเดลพื้นฐานเพื่อใช้ในการดึงคุณลักษณะจากภาพ
- เพิ่มเลเยอร์ใหม่สำหรับการเรียนรู้เฉพาะงาน
- กำหนดเลเยอร์สุดท้ายสำหรับจำแนกภาพจำนวน 119 คลาส
- ตั้งค่าการคอมไพล์โมเดลด้วย Adam และ Categorical Crossentropy
- ใช้ Accuracy เป็นตัวชี้วัดประสิทธิภาพของโมเดล

การสร้างโมเดล InceptionV3

```python
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, ModelCheckpoint

early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=3, min_lr=1e-6)
checkpoint = ModelCheckpoint('best_dog_model.h5', monitor='val_accuracy', save_best_only=True)

history = model.fit(
    train_generator,
    epochs=20,
    validation_data=valid_generator,
    callbacks=[early_stop, reduce_lr, checkpoint]
)
```

2.การฝึกโมเดล

- ใช้ Early Stopping เพื่อหยุดการฝึกอัตโนมัติเมื่อค่า Validation Loss ไม่ดีขึ้น
- ใช้ Reduce Learning Rate เพื่อลดค่า Learning Rate เมื่อโมเดลเริ่มเรียนรู้ช้าลง
- บันทึกโมเดลที่ให้ผล Validation Accuracy ดีที่สุดด้วย Model Checkpoint
- ทำการฝึกโมเดลสูงสุด 20 Epoch พร้อมตรวจสอบผลด้วย Validation Set

ผลลัพธ์การสร้างโมเดล InceptionV3

```
Epoch 10/20
261/261 ━━━━━━━━━━━━━━━━━━━━ 244s 937ms/step - accuracy: 0.7526 - loss: 0.8038 - val_accuracy: 0.8639 - val_loss: 0.4362 - learning_rate: 0.0010
Epoch 11/20
261/261 ━━━━━━━━━━━━━━━━━━━━ 248s 949ms/step - accuracy: 0.7613 - loss: 0.7801 - val_accuracy: 0.8639 - val_loss: 0.4245 - learning_rate: 0.0010
Epoch 12/20
261/261 ━━━━━━━━━━━━━━━━━━━━ 246s 942ms/step - accuracy: 0.7624 - loss: 0.7661 - val_accuracy: 0.8660 - val_loss: 0.4205 - learning_rate: 0.0010
Epoch 13/20
261/261 ━━━━━━━━━━━━━━━━━━━━ 0s 874ms/step - accuracy: 0.7837 - loss: 0.6890WARNING:absl:You are saving your model as an HDF5 file via `model.save
261/261 ━━━━━━━━━━━━━━━━━━━━ 244s 933ms/step - accuracy: 0.7837 - loss: 0.6889 - val_accuracy: 0.8748 - val_loss: 0.3885 - learning_rate: 2.0000e-04
Epoch 14/20
261/261 ━━━━━━━━━━━━━━━━━━━━ 250s 957ms/step - accuracy: 0.8108 - loss: 0.6252 - val_accuracy: 0.8744 - val_loss: 0.3863 - learning_rate: 2.0000e-04
Epoch 15/20
261/261 ━━━━━━━━━━━━━━━━━━━━ 0s 888ms/step - accuracy: 0.8090 - loss: 0.5728WARNING:absl:You are saving your model as an HDF5 file via `model.save
261/261 ━━━━━━━━━━━━━━━━━━━━ 247s 947ms/step - accuracy: 0.8090 - loss: 0.5729 - val_accuracy: 0.8836 - val_loss: 0.3780 - learning_rate: 2.0000e-04
Epoch 16/20
261/261 ━━━━━━━━━━━━━━━━━━━━ 247s 946ms/step - accuracy: 0.8155 - loss: 0.5851 - val_accuracy: 0.8798 - val_loss: 0.3800 - learning_rate: 2.0000e-04
Epoch 17/20
261/261 ━━━━━━━━━━━━━━━━━━━━ 0s 890ms/step - accuracy: 0.8106 - loss: 0.6105WARNING:absl:You are saving your model as an HDF5 file via `model.save
261/261 ━━━━━━━━━━━━━━━━━━━━ 253s 971ms/step - accuracy: 0.8106 - loss: 0.6105 - val_accuracy: 0.8853 - val_loss: 0.3714 - learning_rate: 2.0000e-04
Epoch 18/20
261/261 ━━━━━━━━━━━━━━━━━━━━ 248s 949ms/step - accuracy: 0.8222 - loss: 0.5632 - val_accuracy: 0.8790 - val_loss: 0.3734 - learning_rate: 2.0000e-04
Epoch 19/20
261/261 ━━━━━━━━━━━━━━━━━━━━ 242s 927ms/step - accuracy: 0.8289 - loss: 0.5358 - val_accuracy: 0.8815 - val_loss: 0.3762 - learning_rate: 2.0000e-04
Epoch 20/20
261/261 ━━━━━━━━━━━━━━━━━━━━ 247s 945ms/step - accuracy: 0.8222 - loss: 0.5625 - val_accuracy: 0.8819 - val_loss: 0.3721 - learning_rate: 2.0000e-04
```

ความแม่นยำบนชุดตรวจสอบ (Validation Accuracy) สูงสุดที่ประมาณ 88.53%

- โมเดลให้ผลดีที่สุดในช่วง Epoch ที่ 17
- หลังจากนั้น Validation Accuracy ทรงตัว แสดงว่าโมเดลเริ่มเข้าสู่ภาวะอิ่มตัวในการเรียนรู้
- ระบบได้บันทึกโมเดลที่ให้ผลดีที่สุดไว้สำหรับการนำไปทดสอบและใช้งานต่อ

การสร้างโมเดล EfficienNetB3

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout, BatchNormalization
from tensorflow.keras.applications import EfficientNetB3

base_model = EfficientNetB3(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
base_model.trainable = False

model = Sequential([
    base_model,
    GlobalAveragePooling2D(),
    BatchNormalization(),
    Dense(512, activation='relu'),
    Dropout(0.4),
    Dense(119, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
print()
```

1. การสร้างโมเดล

- เลือกใช้ EfficientNetB3 ที่ผ่านการฝึกจาก ImageNet เป็นโมเดลพื้นฐาน
- แช่แข็งพารามิเตอร์ของโมเดลพื้นฐานเพื่อใช้ในการดึงคุณลักษณะจากภาพ
- เพิ่ม Batch Normalization และ Dense Layer เพื่อเพิ่มประสิทธิภาพการเรียนรู้
- ใช้ Dropout เพื่อลดปัญหา Overfitting
- กำหนดเลเยอร์สุดท้ายสำหรับจำแนกภาพ 119 คลาส
- ตั้งค่าการฝึกด้วย Adam และวัดผลด้วยค่า Accuracy

การสร้างโมเดล EfficienNetB3

```python
from tensorflow.keras.callbacks import ModelCheckpoint, ReduceLROnPlateau

checkpoint = ModelCheckpoint('best_dog_model_B3.keras', save_best_only=True, monitor='val_accuracy', mode='max')
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=2, min_lr=1e-7)
print()

history = model.fit(
    train_gen,
    epochs=5,
    validation_data=valid_gen,
    callbacks=[checkpoint, reduce_lr]
)
```

2. การฝึกโมเดล

- ใช้ Model Checkpoint เพื่อบันทึกโมเดลที่ให้ค่า Validation Accuracy สูงที่สุด
- ใช้ Reduce Learning Rate เมื่อ Validation Loss ไม่ดีขึ้น
- ทำการฝึกโมเดลเป็นจำนวน 5 Epoch พร้อมตรวจสอบผลด้วย Validation Set
- ระบบบันทึกโมเดลที่ดีที่สุดอัตโนมัติสำหรับการนำไปใช้งานต่อ

ผลลัพธ์การสร้างโมเดล EfficienNetB3

```
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**kwargs
  self._warn_if_super_not_called()
Epoch 1/5
261/261 ━━━━━━━━━━━━━━━━━━━━ 6299s 24s/step - accuracy: 0.5182 - loss: 2.2554 - val_accuracy: 0.8441 - val_loss: 0.5197 - learning_rate: 0.0010
Epoch 2/5
261/261 ━━━━━━━━━━━━━━━━━━━━ 1693s 6s/step - accuracy: 0.7753 - loss: 0.8351 - val_accuracy: 0.8462 - val_loss: 0.6136 - learning_rate: 0.0010
Epoch 3/5
261/261 ━━━━━━━━━━━━━━━━━━━━ 1682s 6s/step - accuracy: 0.8194 - loss: 0.6761 - val_accuracy: 0.8546 - val_loss: 0.5991 - learning_rate: 0.0010
Epoch 4/5
261/261 ━━━━━━━━━━━━━━━━━━━━ 1701s 7s/step - accuracy: 0.8499 - loss: 0.5382 - val_accuracy: 0.8727 - val_loss: 0.5043 - learning_rate: 2.0000e-04
Epoch 5/5
261/261 ━━━━━━━━━━━━━━━━━━━━ 1682s 6s/step - accuracy: 0.8728 - loss: 0.4245 - val_accuracy: 0.8727 - val_loss: 0.4894 - learning_rate: 2.0000e-04
```

ความแม่นยำบนชุดตรวจสอบ (Validation Accuracy) สูงสุดที่ประมาณ 87.27%

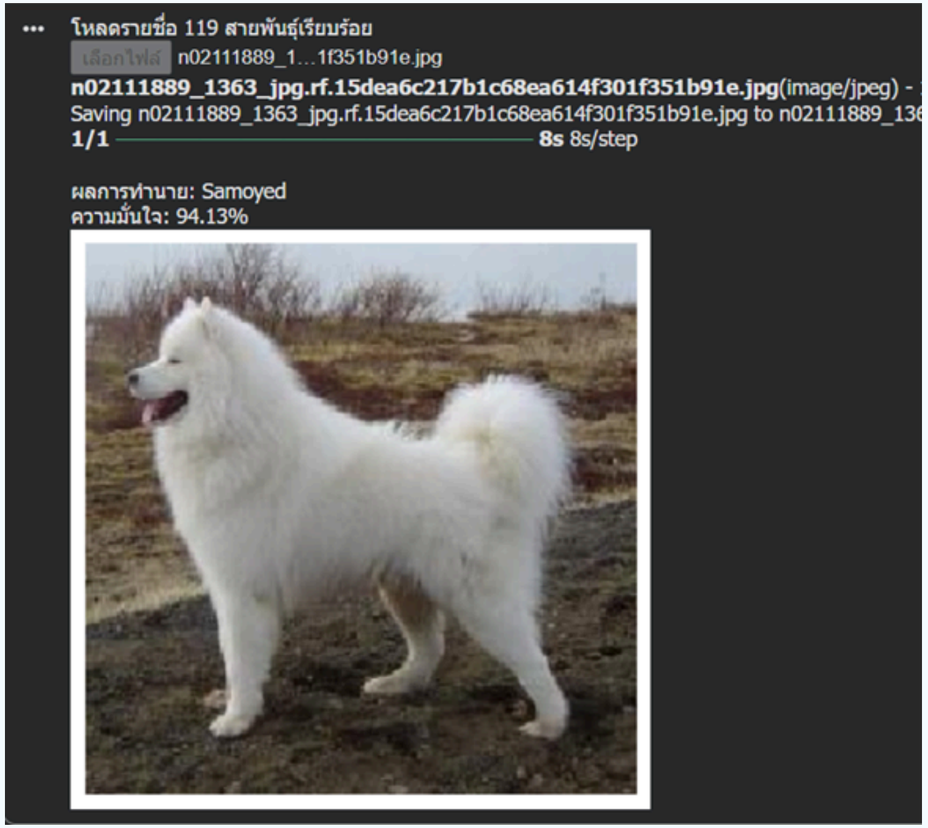
- โมเดลให้ผลดีที่สุดในช่วง Epoch ที่ 4
- หลังลด Learning Rate โมเดลมีประสิทธิภาพดีขึ้นอย่างชัดเจน
- ระบบได้บันทึกโมเดลที่ให้ผลดีที่สุดไว้สำหรับการนำไปทดสอบและใช้งานต่อ

การทำนายสายพันธุ์สุนัขจากโมเดล MobileNetV2



นำข้อมูลภาพจากโฟลเดอร์ Test set โดยเลือกใช้ภาพสุนัขสายพันธุ์ Samoyed เพื่อทดสอบประสิทธิภาพของโมเดล ผลการทดสอบพบว่าโมเดลแสดงค่าความมั่นใจ (Confidence) ในการทำนายผลเท่ากับ 96.54%

การทำนายสายพันธุ์สุนัขจากโมเดล InceptionV3



นำข้อมูลภาพจากโฟลเดอร์ Test set โดยเลือกใช้ภาพสุนัขสายพันธุ์ Samoyed เพื่อทดสอบประสิทธิภาพของโมเดล ผลการทดสอบพบว่าโมเดลแสดงค่าความมั่นใจ (Confidence) ในการทำนายผลเท่ากับ 94.13%

การทำนายสายพันธุ์สุนัขจากโมเดล EfficientNetB3



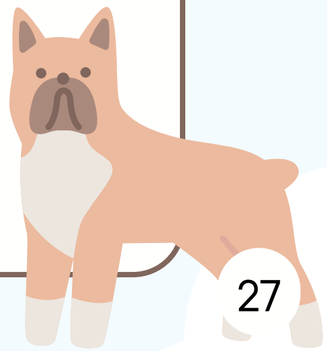
นำข้อมูลภาพจากโฟลเดอร์ Test set โดยเลือกใช้ภาพสุนัขสายพันธุ์ Samoyed เพื่อทดสอบประสิทธิภาพของโมเดล ผลการทดสอบพบว่าโมเดลแสดงค่าความมั่นใจ (Confidence) ในการทำนายผลเท่ากับ 100%

สรุป

Model Architecture	Training Acc (%)	Validation Acc (%)	Training Loss	Validation Loss
MobileNetV2	95	79.81	0.2109	0.6486
InceptionV3	81.06	88.53	0.6105	0.3714
EfficientNetB3	87.28	87.27	0.4245	0.4894

จากการทดลองพบว่า InceptionV3 คือโมเดลที่มีประสิทธิภาพดีที่สุด โดยทำคะแนนความแม่นยำบนชุดข้อมูลทดสอบได้สูงถึง 88.53% และมีค่าความสูญเสียต่ำที่สุด แสดงถึงความสามารถในการดึงเอกลักษณ์ของภาพมาวิเคราะห์ได้อย่างแม่นยำ ในขณะที่ EfficientNetB3 โดดเด่นในด้านความเสถียรของโมเดล (Consistency) และ MobileNetV2 แม้จะเรียนรู้ได้รวดเร็วแต่พบปัญหาการเรียนรู้เกิน (Overfitting) ทำให้ประสิทธิภาพในการใช้งานจริงยังไม่สูงเท่าโมเดลอื่น





Thank You
For Attention